KING MONGKUT'S INSTITUTE OF TECHNOLOGY LATKRABANG
SCHOOL OF ENGINEERING
GROUP OF (**ROBOTICS & AI : Section 2)**

01416516 - ROBOTICS LABORATORY 3

LAB TITLE: INTRO TO TINYML

INSTRUCTED BY: Aj.JAMES

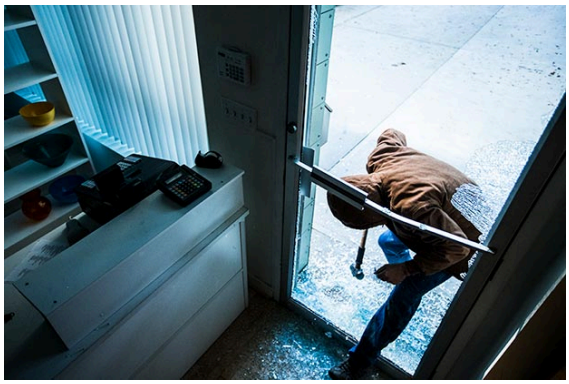
NAME: THURA, SORNPAWEE, SWAN
ID NUMBER: 65011685, 65011546, 65011683
GROUP NO: 4 - RAI2
DATE OF SUB. 29/1/2024

Table of contents

Abstract

Vandalism Prevention

In recent years, vandalism crimes have reached alarming levels, posing serious threats to public safety and property security. The severity of these incidents extends beyond mere property damage, often causing emotional distress and financial burdens for victims. To tackle this growing issue, the integration of AI technology emerges as a promising solution. By leveraging advanced machine learning algorithms, specifically tailored for vandalism detection, we can enhance our ability to prevent and respond to these crimes. AI-driven surveillance systems, equipped with real-time analysis of audio and visual cues, offer a proactive approach to identify potential acts of vandalism, such as **glass breakage** or suspicious activities.

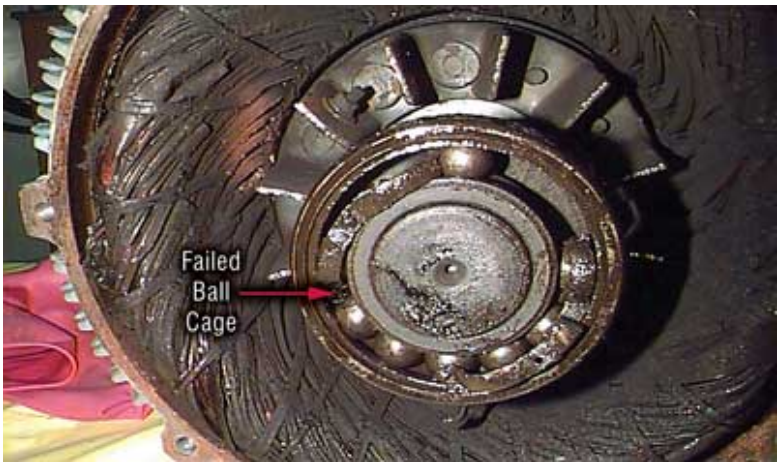


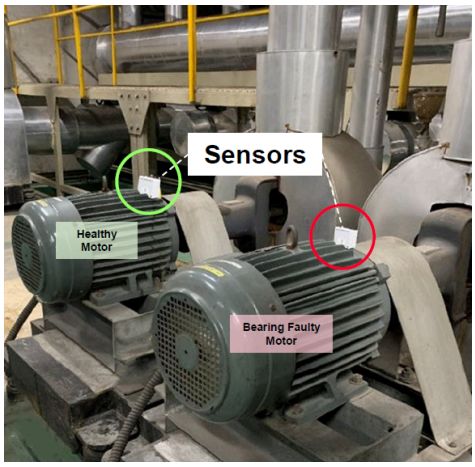
Industrial Safety

The detection of **faulty motors** in industrial settings holds paramount importance as it directly impacts operational efficiency, production quality, and overall safety. Motors are integral components across diverse industrial sectors, powering machinery and equipment critical to manufacturing processes. When motors fail unexpectedly, it can result in costly downtime, production delays, and substantial financial losses. Additionally, equipment breakdowns can pose safety risks to workers and compromise product quality. Recognizing

the critical role that motors play in industrial operations, the integration of AI-powered sensors emerges as a proactive solution to prevent such failures.

AI-powered sensors equipped with machine learning algorithms can continuously monitor the performance of motors in real-time. These sensors analyze data patterns, vibrations, and other operational parameters to detect subtle anomalies indicative of potential faults or impending failures.In essence, the use of AI-powered sensors for detecting faulty motors represents a strategic investment in maintaining a resilient and sustainable industrial infrastructure.





In this report, we will be creating two AI models that can solve the above problems.

1. **Vocal Model** that can detect glass breaking sound and ‘help me’ phrase
2. **Vibration Model** that can classify healthy vibrations and faulty vibrations of motors.

In order to achieve accurate results, we need to carefully choose **correct setups** of experiments.

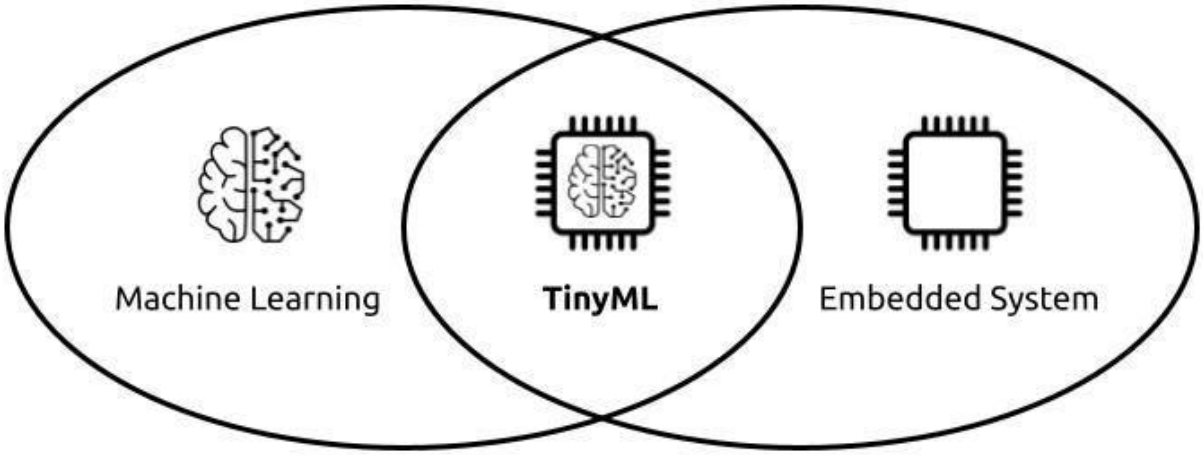
Introduction

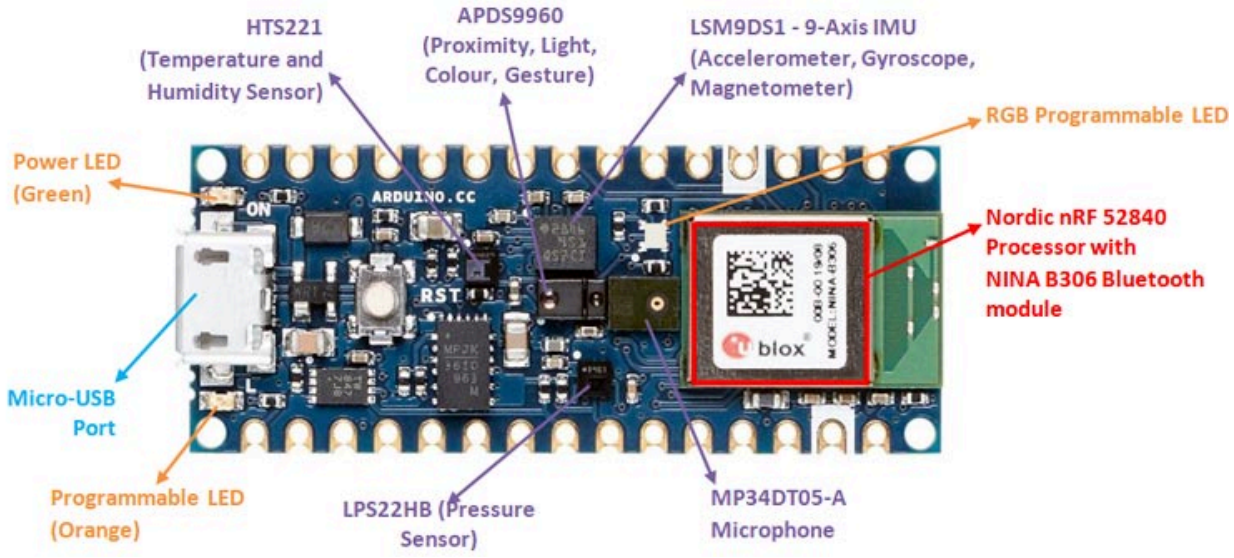
Hardware Selection

In these experiments, hardwares should be able to:

1. record and process audio samples
2. detect a different type of vibrations

For that reason, we are given a special development board called **Arduino Nano 33 BLE** which supports tinyML capabilities.

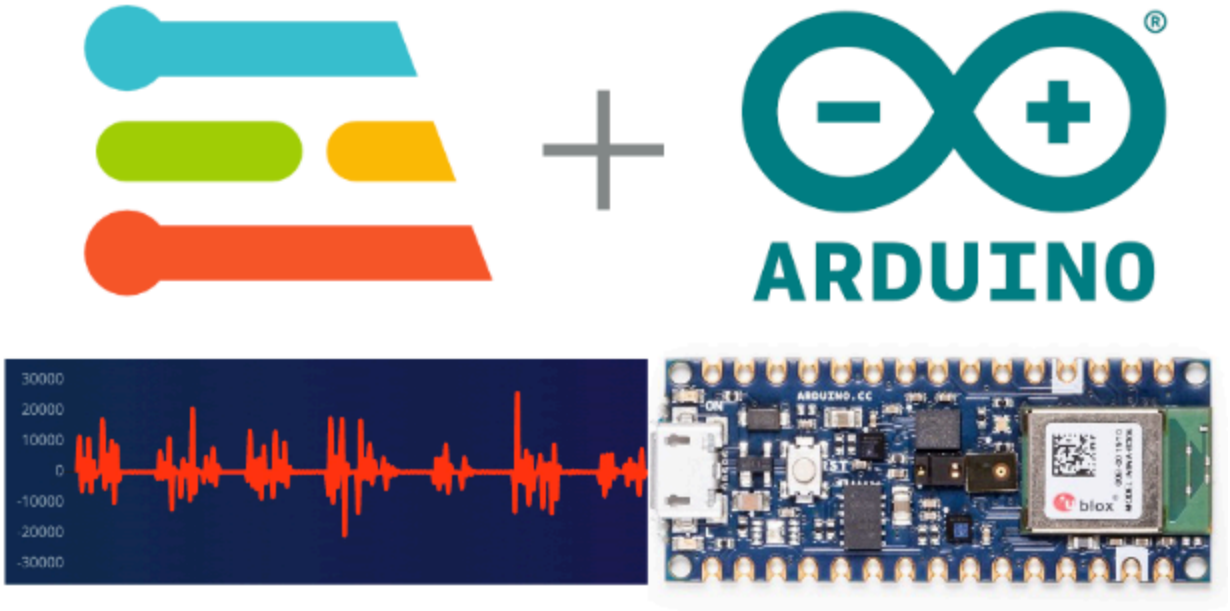




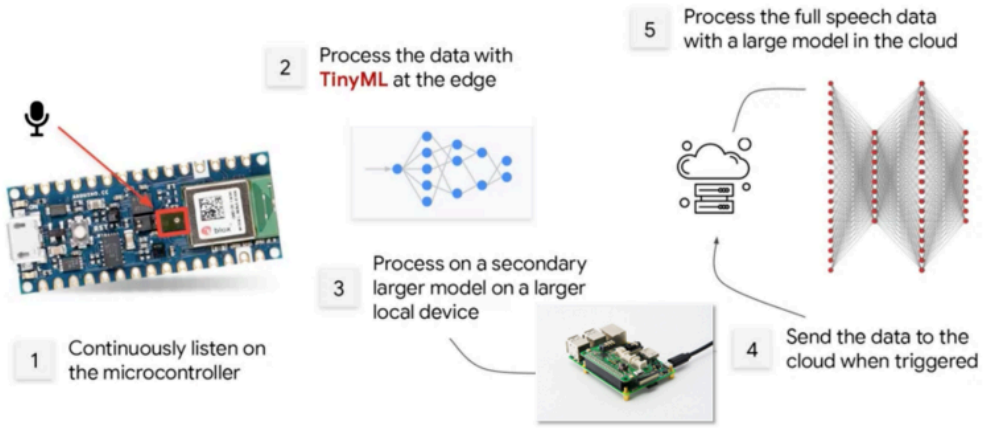
The Arduino Nano 33 IoT comes with an **IMU** (LSM6DS3), Accelerometer and Gyroscope -For motion sensing and gesture recognition, **Microphone** (MP34DT05): For sound and voice recognition, Environmental Sensors (e.g., temperature, humidity, pressure). The board comes with **Wi-Fi connectivity** and Arduino Cloud compatibility. The Nano 33 IoT is Bluetooth enabled allowing you to control peripheral devices via Bluetooth and start implementing Bluetooth low energy applications.

Software Environment

Creating a machine learning model from scratch and training on our own is not an easy task. Due to time and budget constraints, we used an educational tool called **Edge Impulse** which was created with the purpose of making Machine Learning **easier.**



Additionally, after we get our accurate model, we need to deploy to Arduino by compiling **human readable source code to machine binary code using Arduino IDE**. Furthermore, Arduino IDE's capability to access the **Serial Monitor** tool accelerates the development process even more.



Implementation

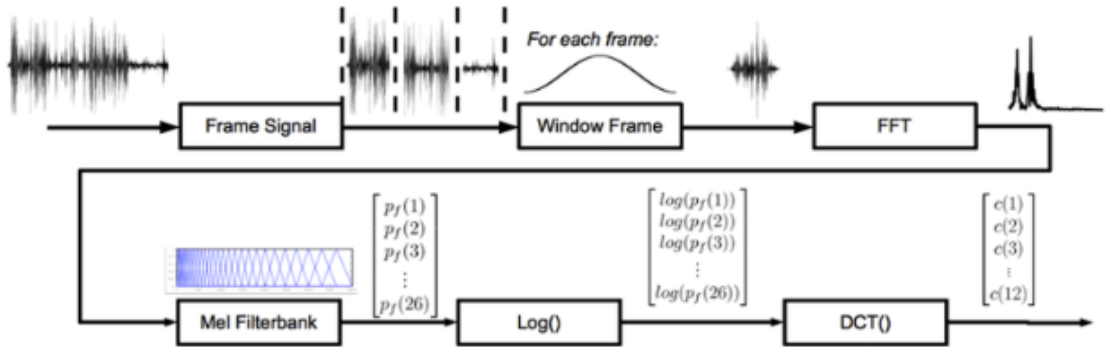
Each experiment(vocal and vibration) underwent the following processes in order to convert theories to real working results.

1. Theoretical Approach
2. Data Collection
3. Parameter tuning and training
4. Validate the model using test data
5. Deploy to the hardware and test the new incoming data from foreign source.

Vocal Model

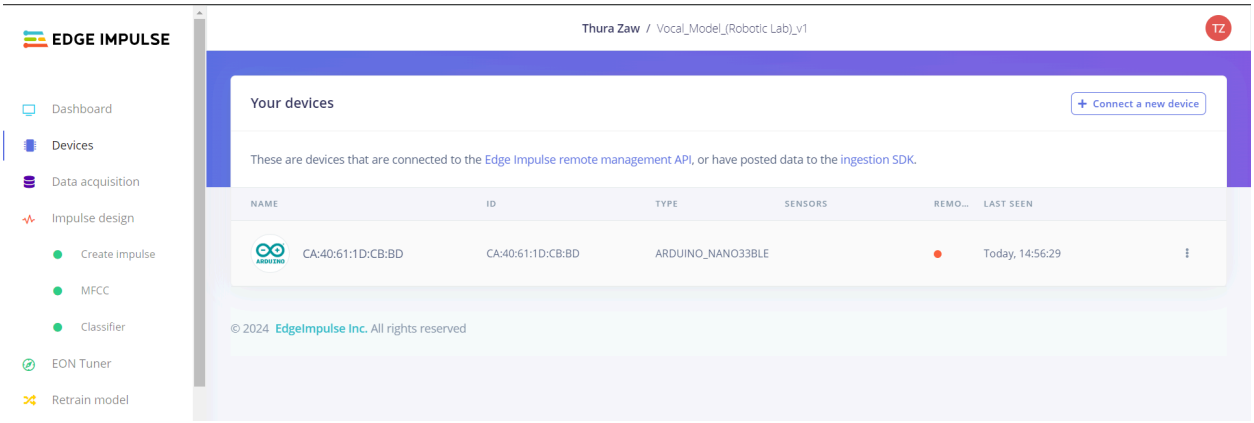
Theory

Mel-Frequency Cepstral Coefficients (MFCC) are a crucial feature extraction technique in voice recognition systems. This process involves pre-emphasis, frame blocking, windowing, Fast Fourier Transform (FFT), Mel filtering, logarithmic compression, and Discrete Cosine Transform (DCT). The MFCCs represent the spectral characteristics of audio signals in a way suitable for machine learning analysis. By capturing essential features of speech signals and reducing dimensionality, MFCCs provide an effective method for tasks such as speech recognition and speaker identification. The resulting feature vector is robust against pronunciation variations, making it widely used in voice and speech processing applications.



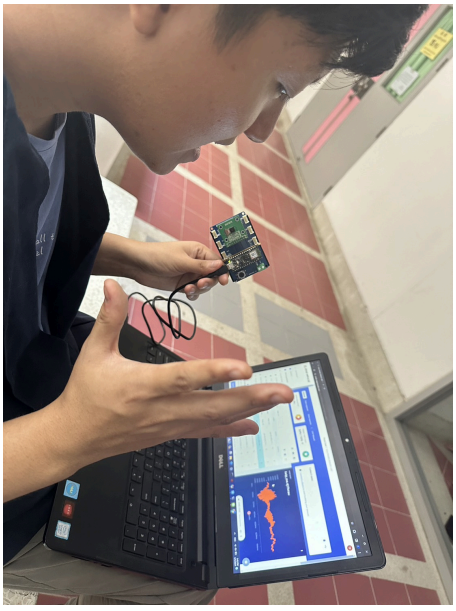
Data Collection

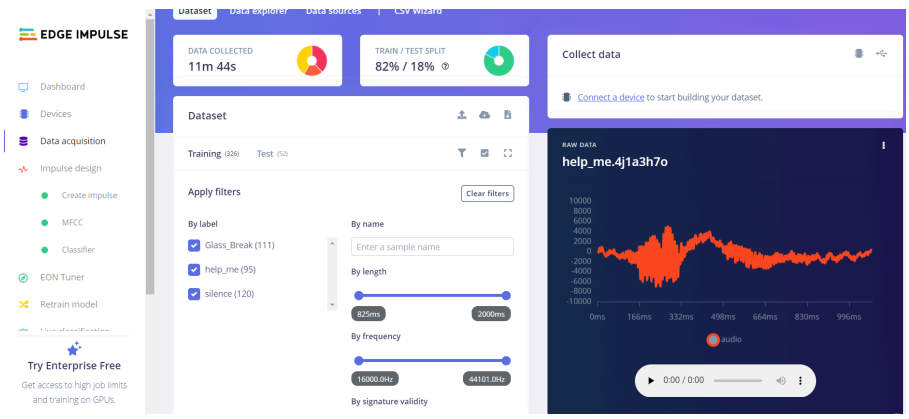
Before we collect the data, we have to connect our board which is an Arduino Nano 33 in Edge Impulse.

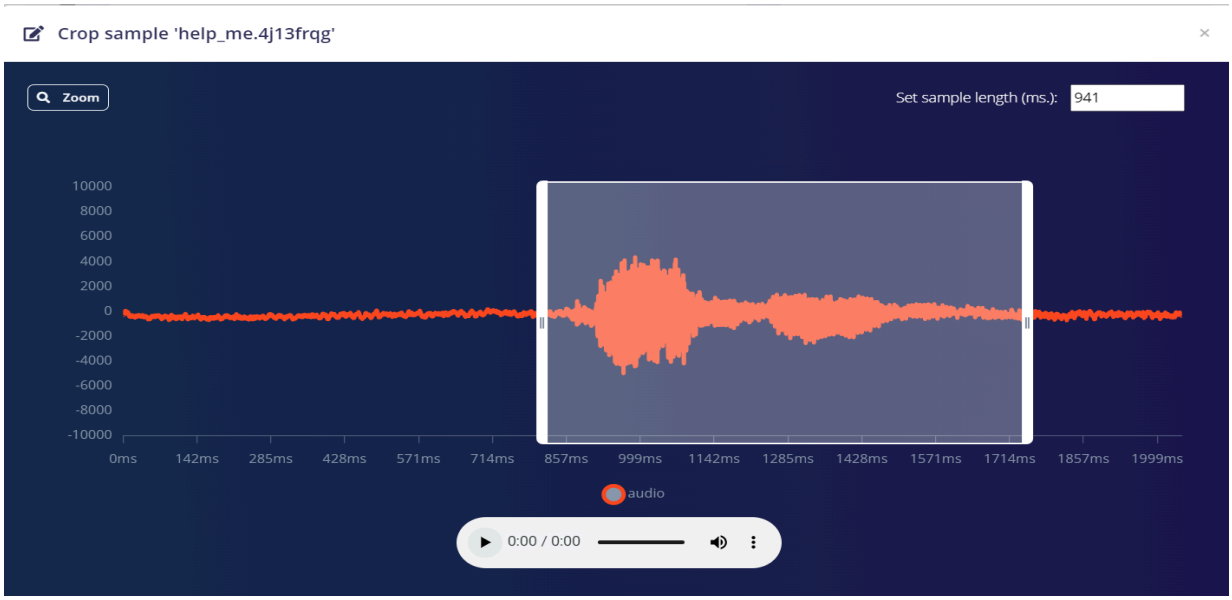


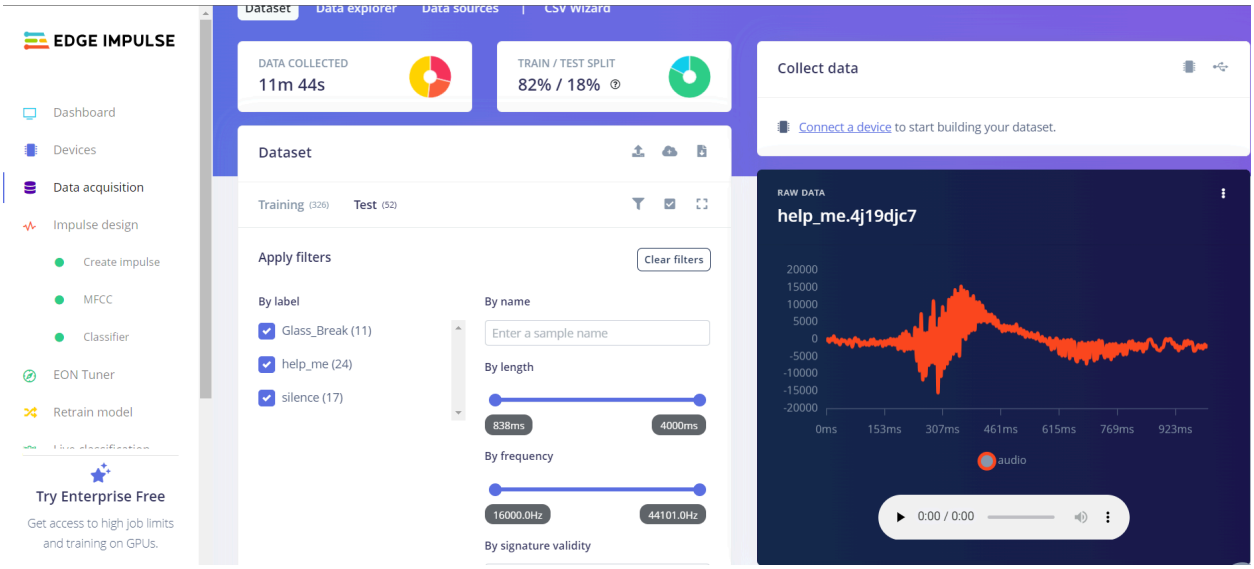
We collected three kinds of sounds which are “Silence”, “Help me” and Glass Breaking. For the silence sound training part, we collected surrounding sound for about 3 seconds and assigned that as Silence. When tested any sound coming up other than the assigned or trained sounds it will recognize as “Silence”. Then we collected two more voice data, “Help me” and “Glass Breaking”. Unlike the “Silence”, “Help me” and “Glass Breaking” acquired around one second and cropped the voice line from the data sample.

We trained all the voice models totaling 326 data in the Data Set. To be specific, 111 “Glass Breaking” data, 95 “Help me” data and 120 “Silence” data.





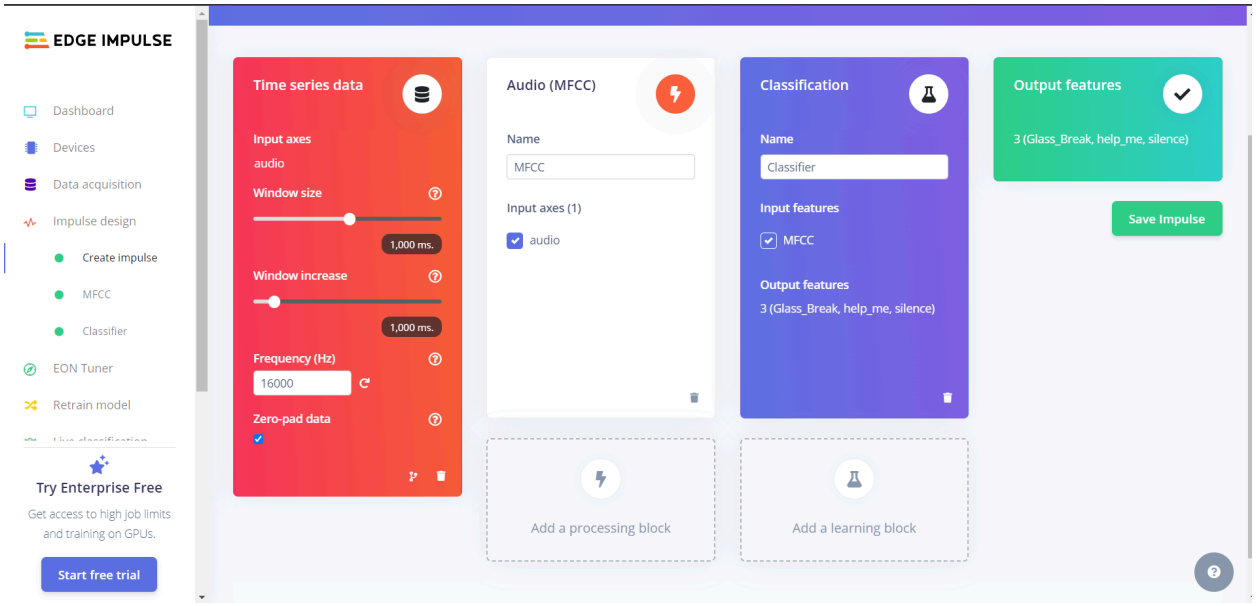


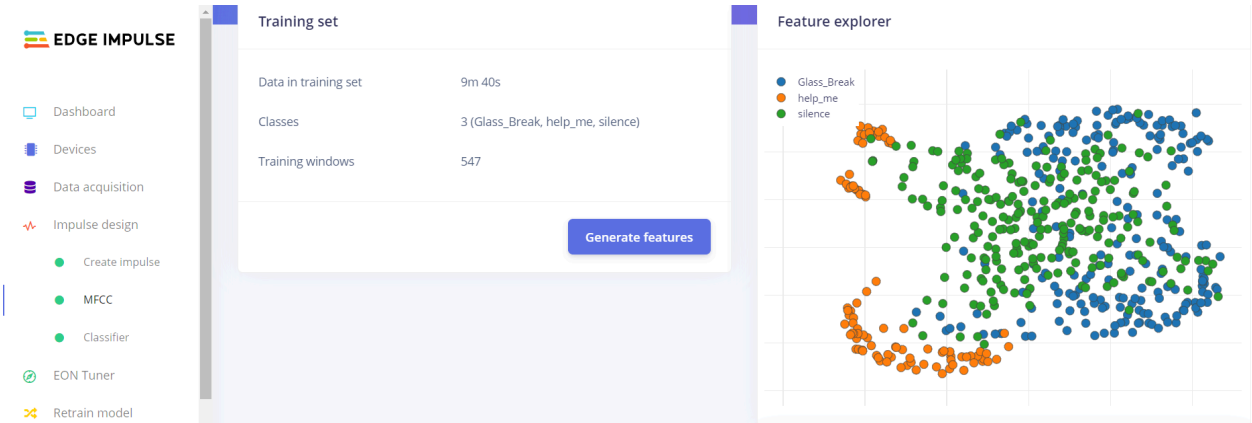


Then , we move some data from the training data set to the test data set. There are totally 52 data in the training data set. So you can say that the train and test split is 82% / 18%.(80/20)

Parameter Tuning

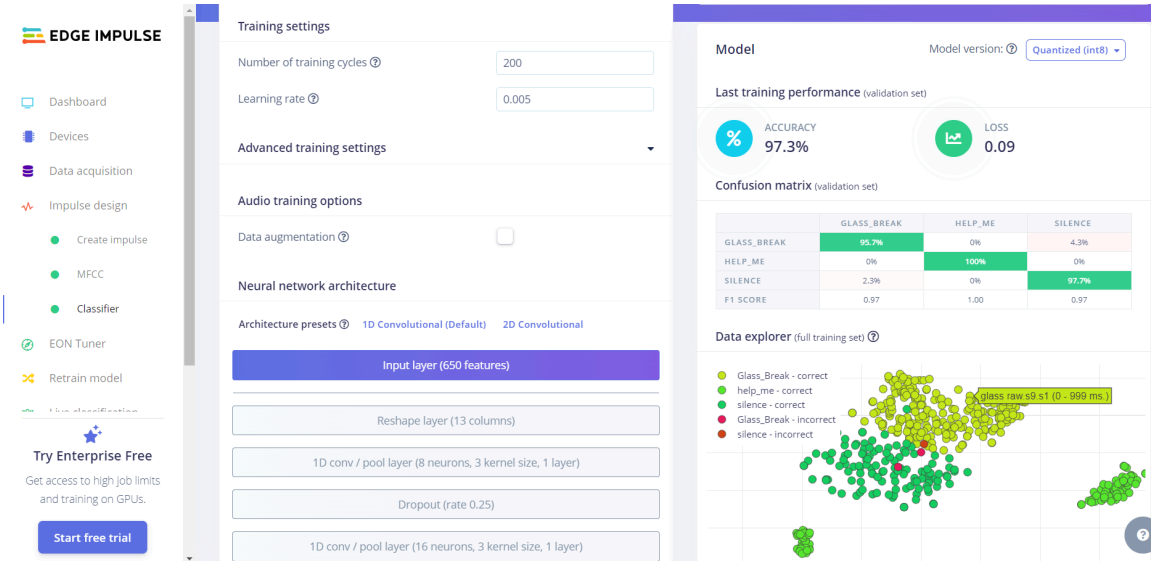
Since each ‘help me’ and ‘glass break’ took around **1.5 seconds**, we configured our model to collect each 1 sec sample from our training set. After we choose appropriate **MFCC** parameters using **Impulse Design,** our sampled voice data are passed into a pre-defined neural network.



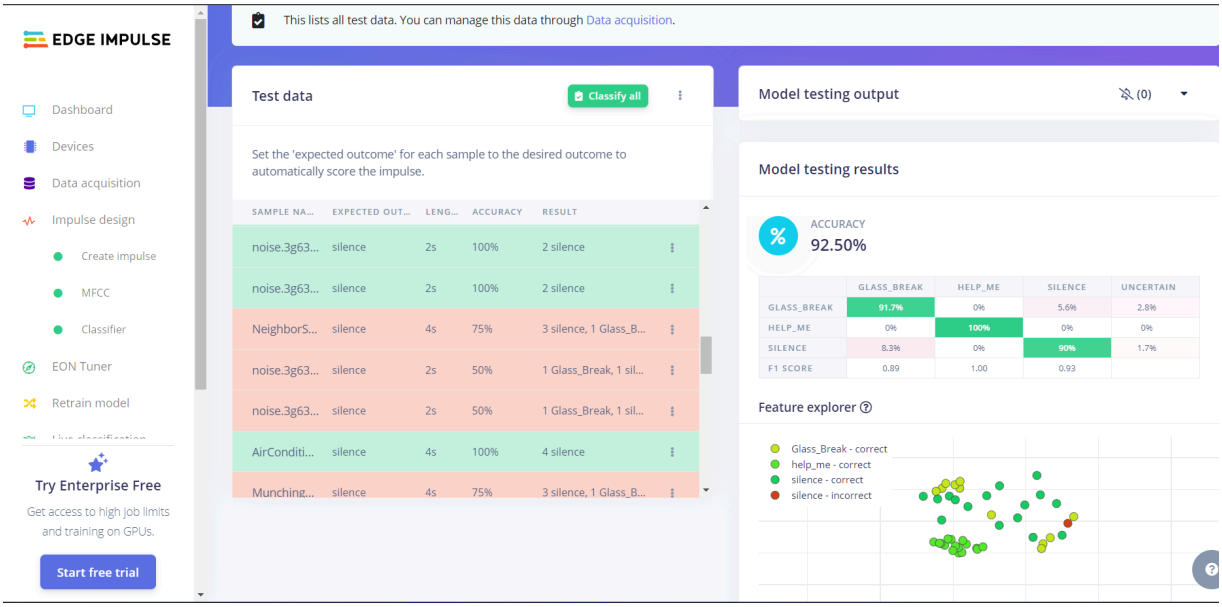


Training Data

While training a neural network, it is important to choose reasonable **learning rate** and **iterations** to minimize the error and to improve model accuracy. That is why we chose 200 iterations and learning rate of 0.005 **to decrease the overshoot in gradient descent**



Model Testing

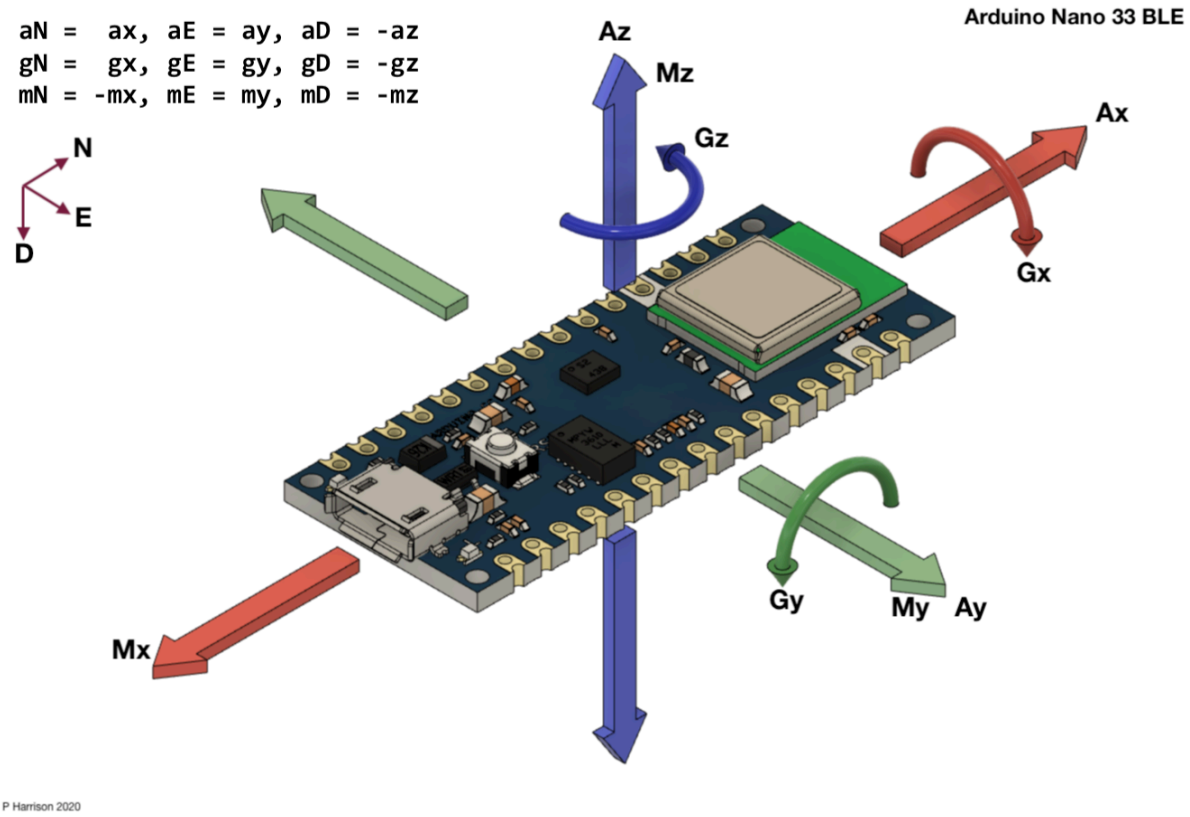


Accuracy of over 90% seems like a **pretty confident result** to us. We are satisfied with this outcome and ready to deploy to Arduino to see how it functions in real world.

Vibration Model

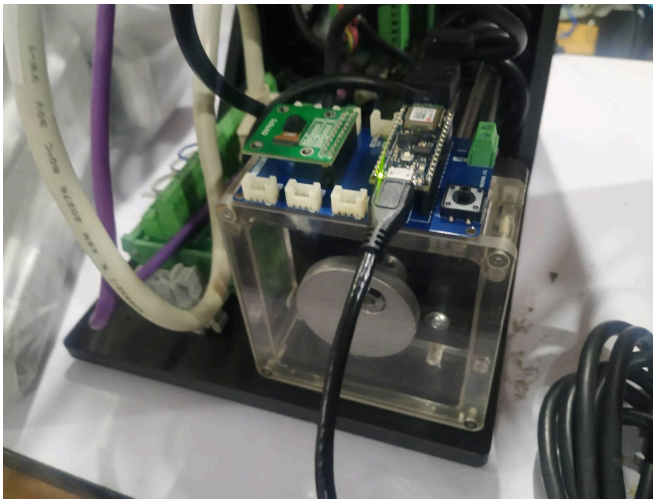
Theory

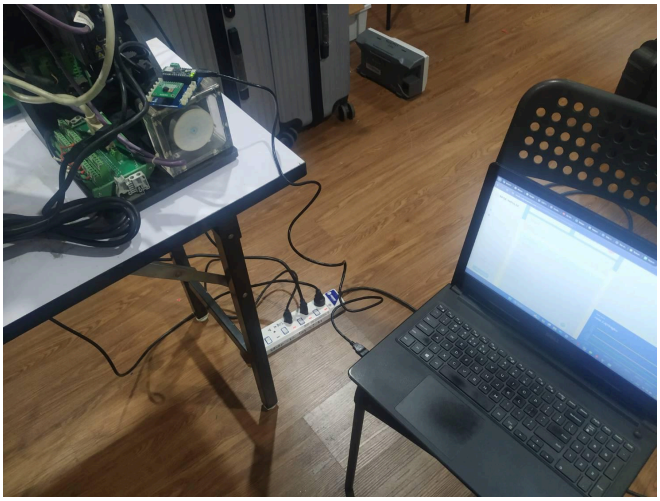
The question is simple. **How to detect vibration**? It is all about the **degree of freedom**. A rigid body in space can theoretically move 6 degrees of freedom. Since vibration is a form of rapid **Z-axis movement**, we can use gyroscopic measurements to classify **different Z-axis amplitudes** of vibrations using IMU (Inertial Measure Unit).

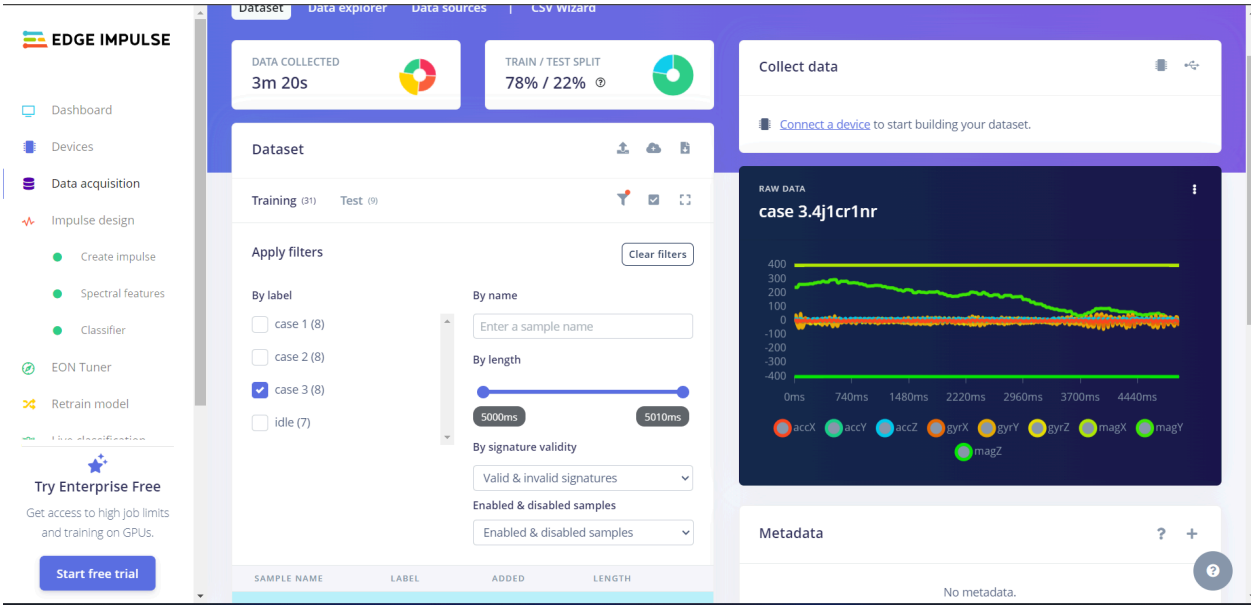


Data Collection

For demonstration purposes, we have three different motors vibrating at different frequencies. Our task is to collect data samples from each motor and create 4 classes.**(case 1, case 2, case 3 case 4, idle)**. We were satisfied with 10 five-seconds samples from each case and continued to work on it.



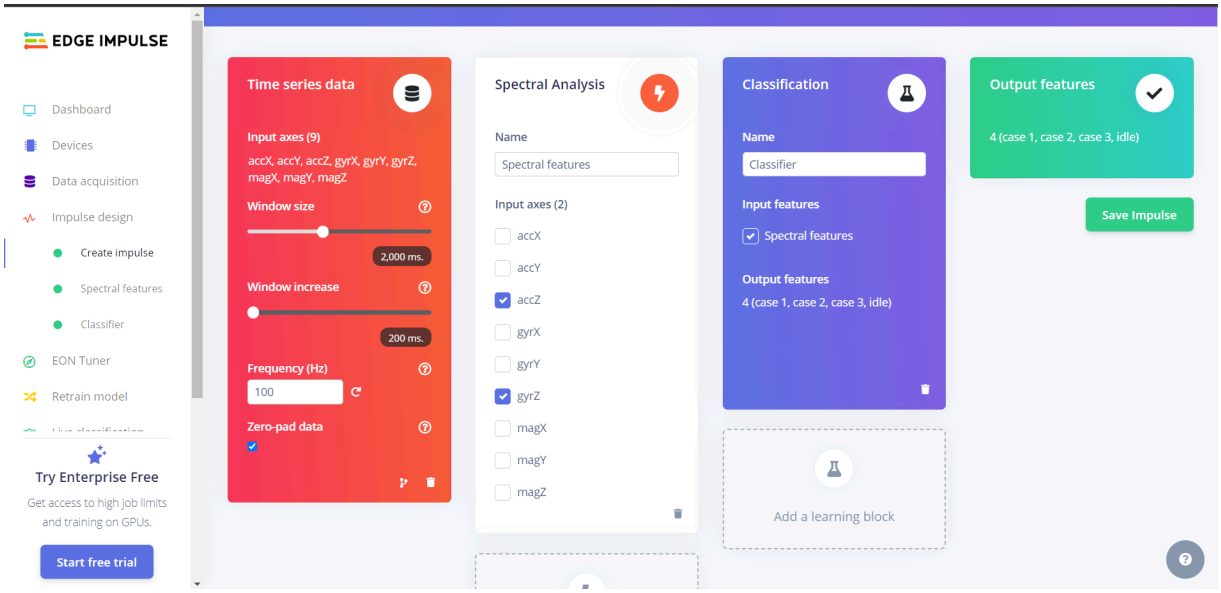


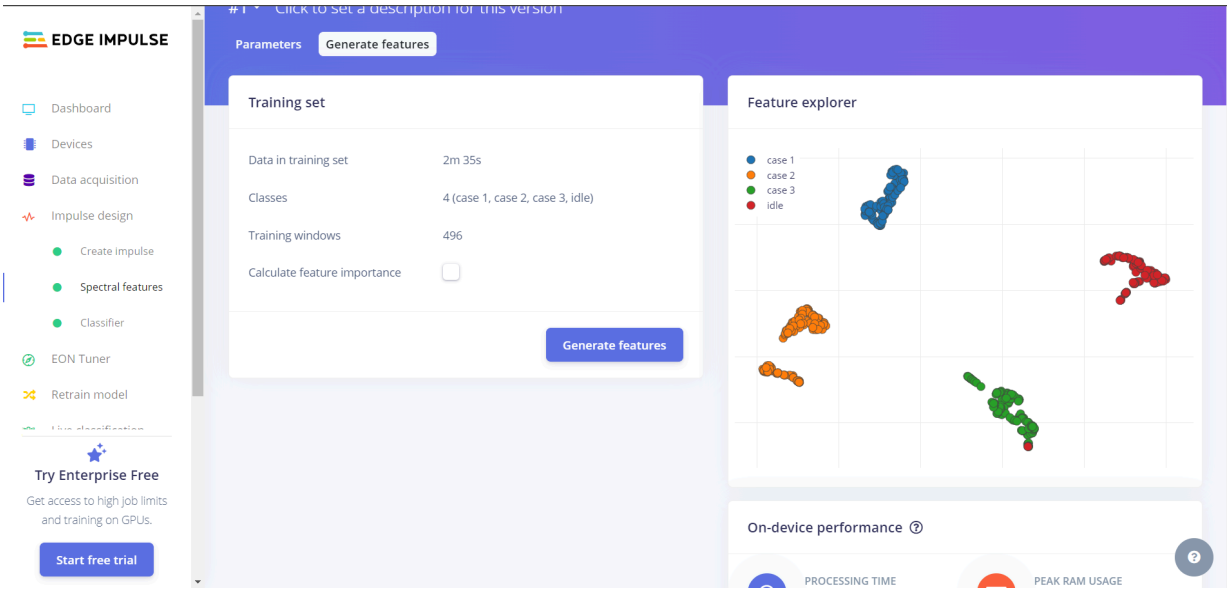


After collecting all vibration data, it is necessary to split into training sets and test sets accordingly. (80/20) split is a good option for most applications..

Parameter Tuning

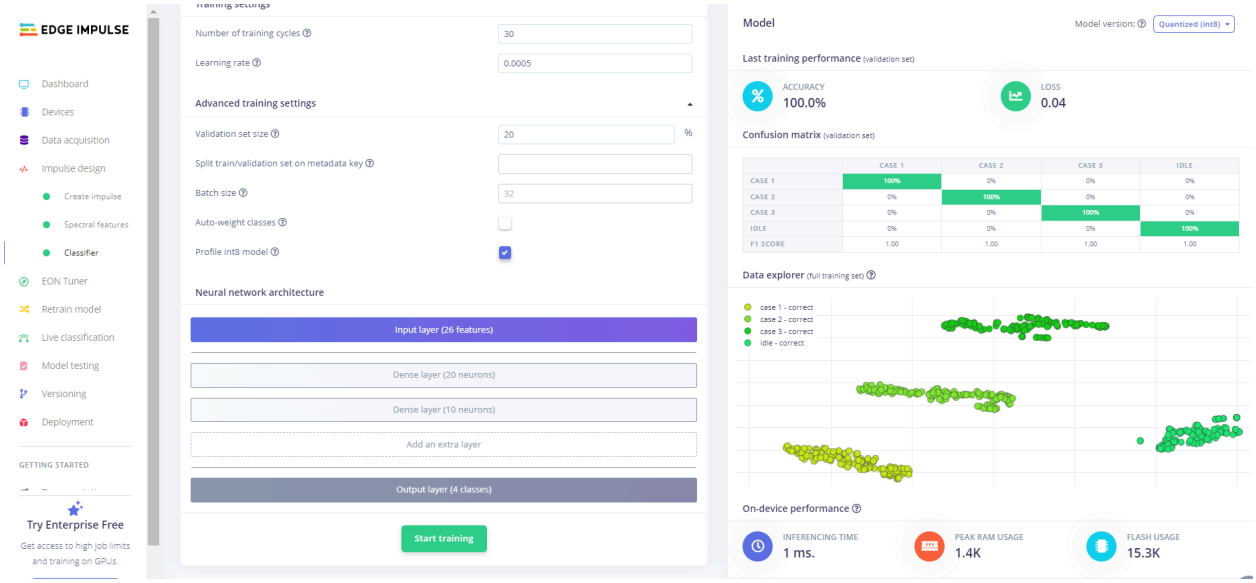
This is where things get very interesting. IMU measures the 9-axis movement of an object.(Gyro, Accelerometer, Magnetometer). Since our experiment does not concern any magnetism, the magnetometer feature was **discarded** from the impulse design. Additionally, we are only interested in the Z-axis transform of the sensors. So, only Z-axis measurements will be saved to impulse design.





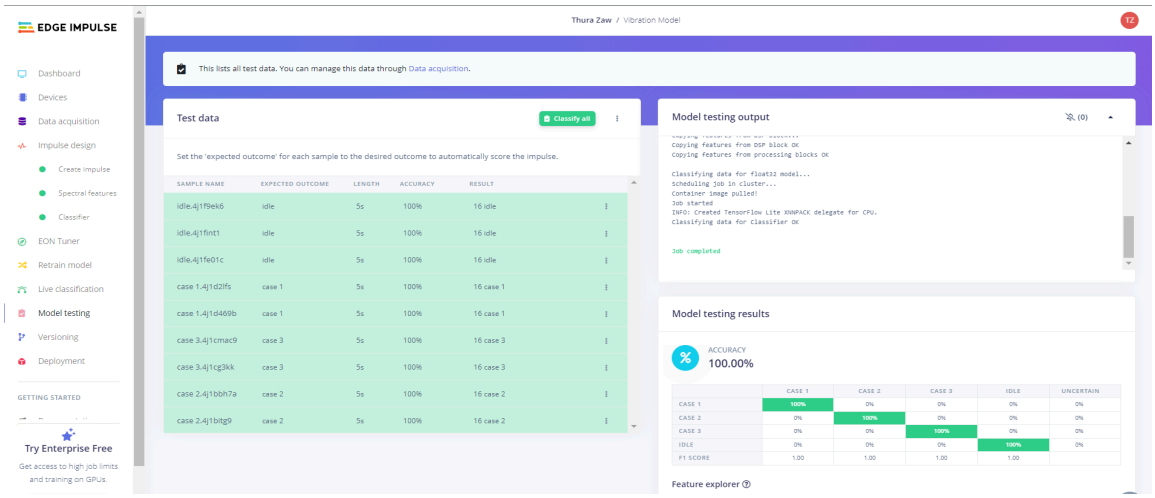
Training Data

Training the vibration model is the same as the previous vocal model. We just need to tune learning rate and iterations to get accurate results. Here is how we trained the model.



We can see that the vibration model's accuracy is better than the vocal model's. It is because the vibration model, unlike vocal, is consistent throughout the data collection since it takes place in a steady environment. How about we try testing our model with the testing set?

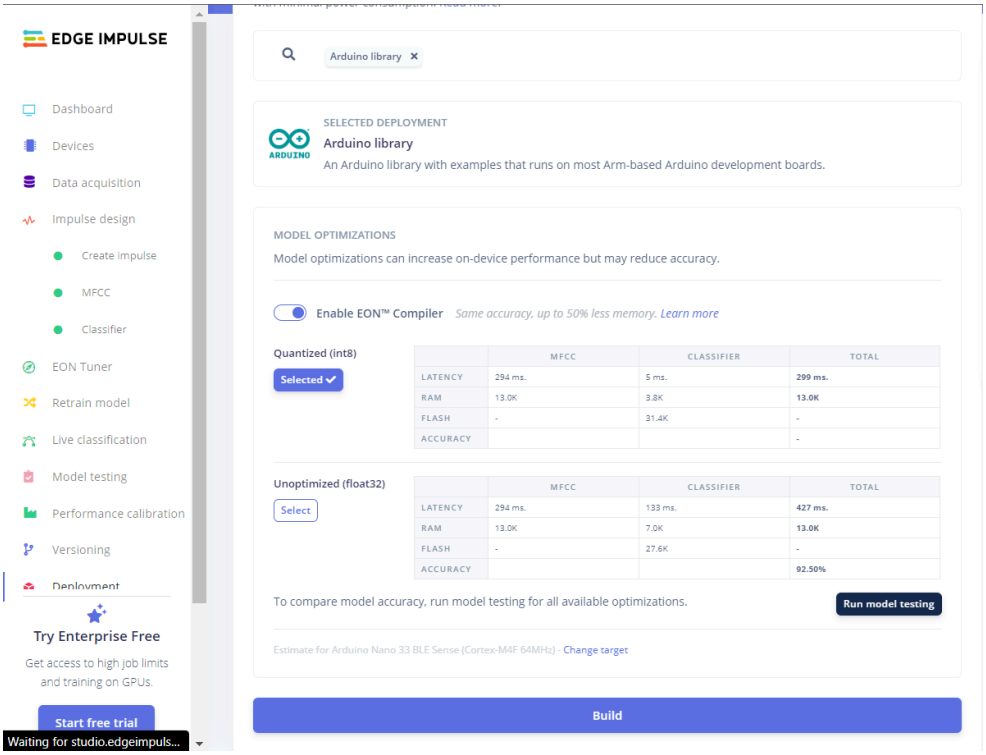
Model Testing

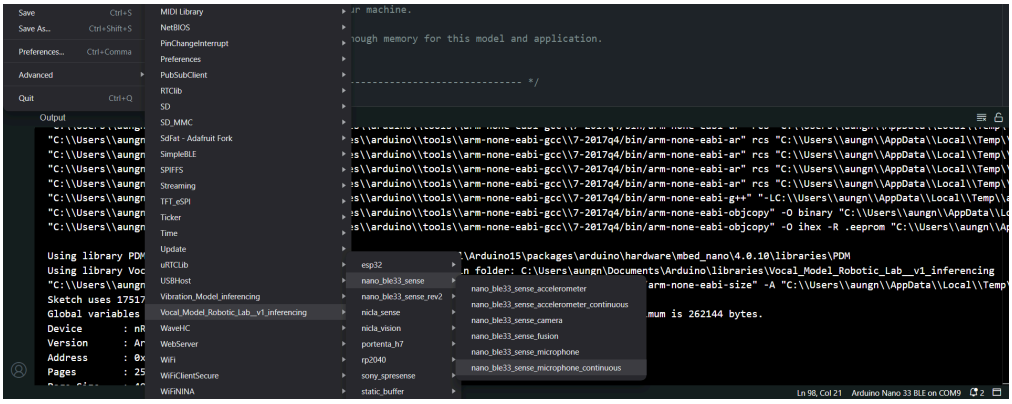


Optimization

Deployment

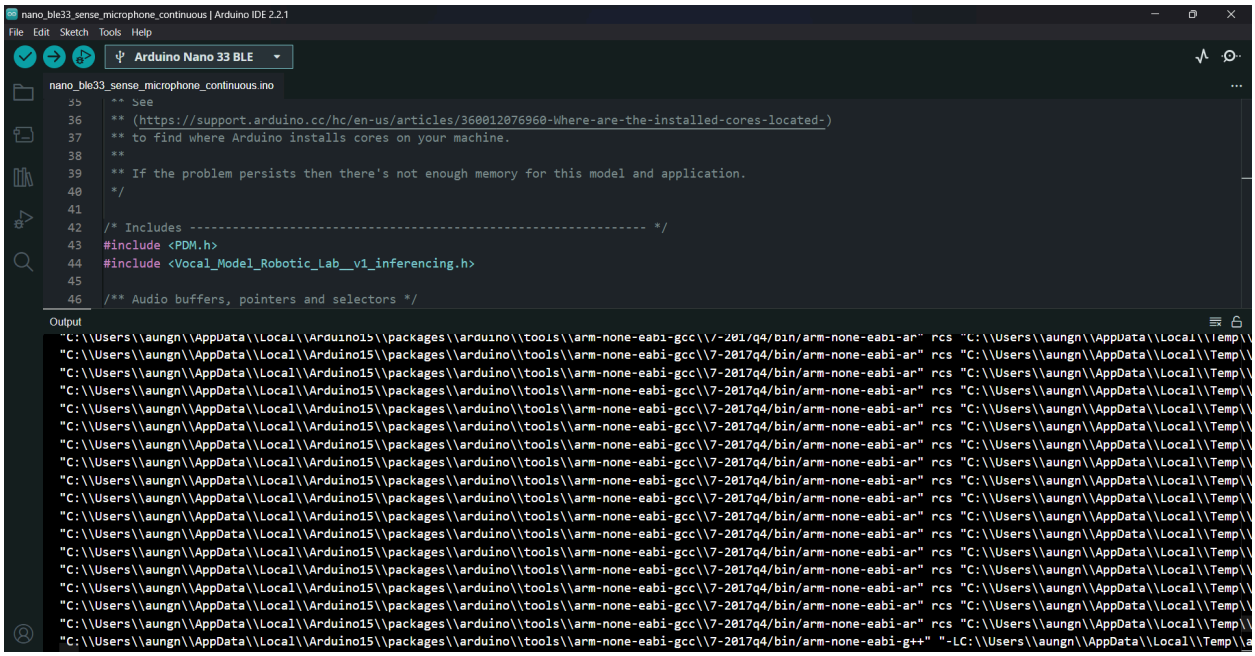
After we are satisfied with our models' accuracy, it is time to deploy both models to the hardware.
Edge Impulse supports several options to export our model. One of them is as in **Arduino Library** for embedded devices which supports sufficient RAM and hardware capabilities for tinyML applications.



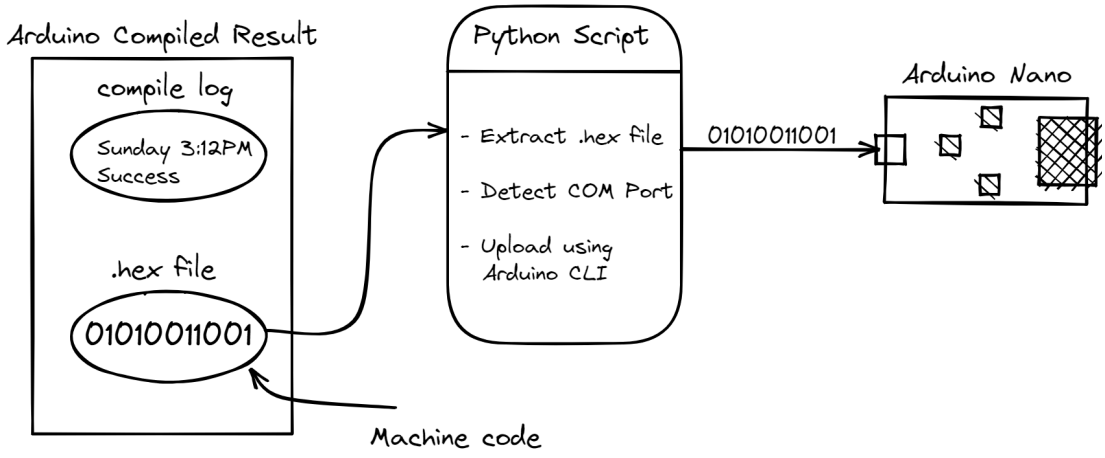


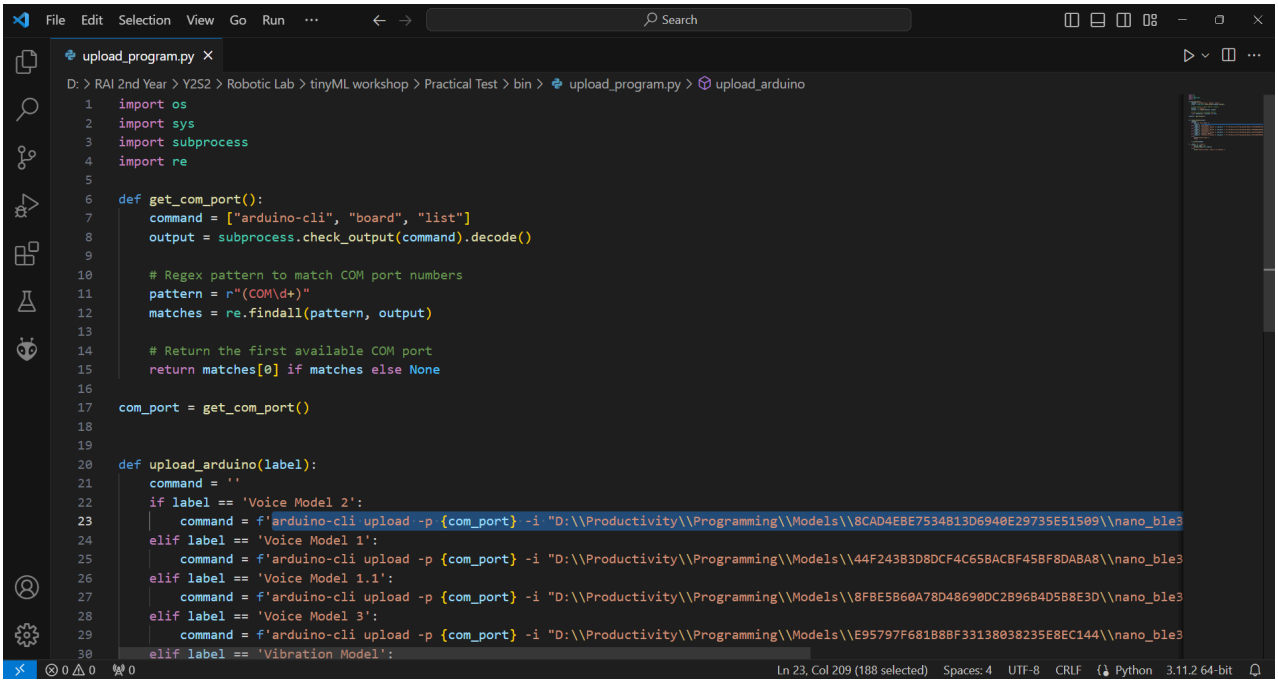
Compile Once, Run Anytime

As we all are aware that compiling and uploading exported Edge Impulse source code in Arduino IDE **takes about a 30 minutes minimum**. Sometimes, we fall asleep while waiting for the process to finish. Arduino IDE **recompiles** the source code even when we only need uploading.

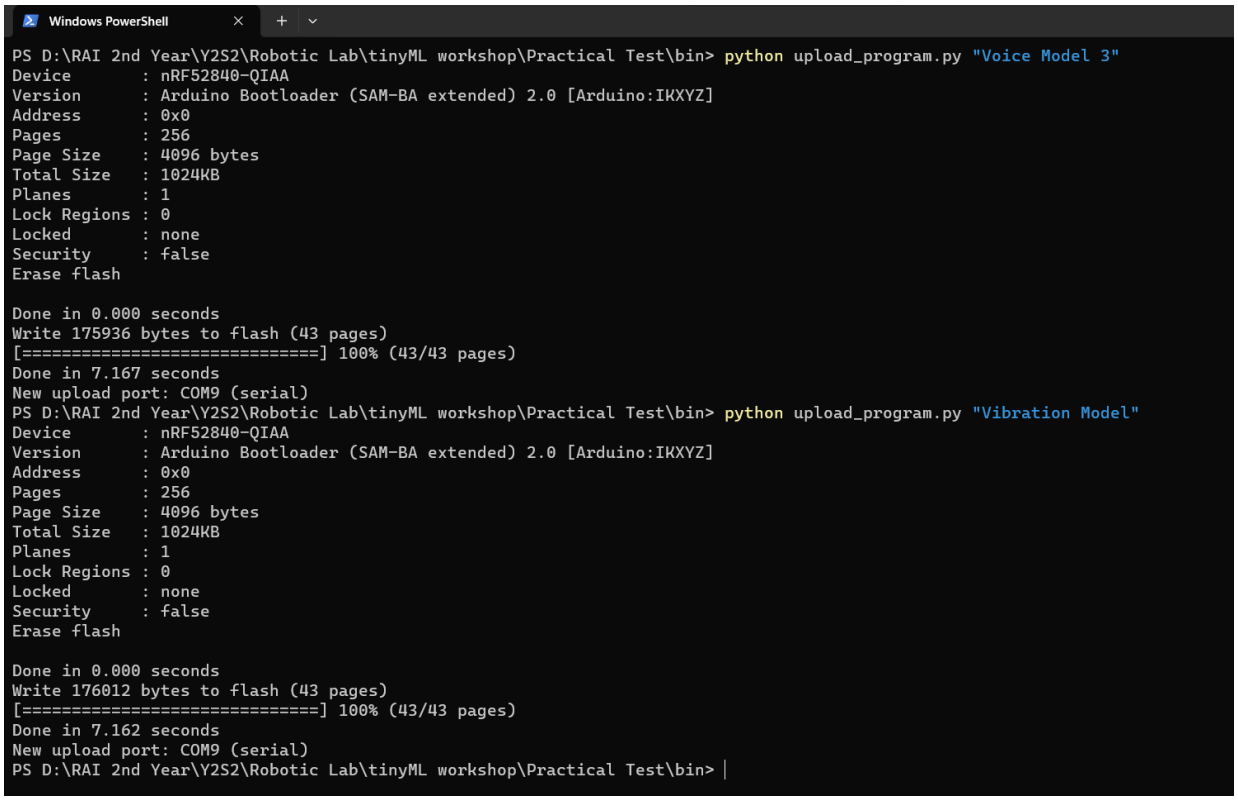


For that reason, one of our group members, Thura Zaw, created a custom python script that handles the compilation and uploading process **separately**. Here is how he did this miracle.





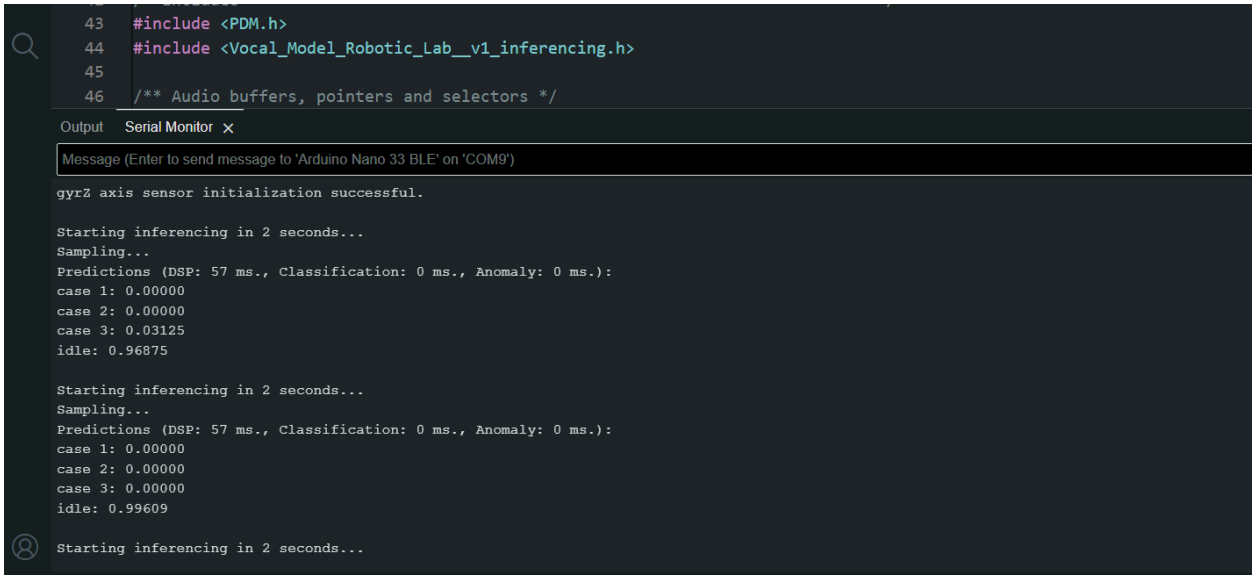
So, we **don't need to recompile** the source code again. All we need to do is connect the board with the computer and run the python script, it will automatically upload the compiled machine code to your hardware.



Results

In order to monitor and debug the results, we use Arduino IDE's built-in Serial Monitor.

Vocal Model Test(Link to Test Video)



Vibration Model Test(Link to Test Video)

Discussion

Implemented voice and vibration models on Arduino Nano 33 BLE Sense using Edge Impulse:

1. Voice Model:
    - Collected labeled voice data, trained with high accuracy.
    - Successfully deployed on Arduino Nano 33 for real-time voice command recognition.
2. Vibration Model:
    - Used accelerometer and gyroscope data to train a model to distinguish normal and abnormal vibrations.
    - Deployed on Arduino Nano 33 for detecting irregularities in machinery.
3. Real-World Applications:
    - Voice model applicable to smart home voice commands.
    - Vibration model aids in predictive maintenance in industrial settings.

References

- Gallup. (2023, October 26). More Americans see the U.S. crime problem as serious. https://news.gallup.com/poll/544442/americans-crime-problem-serious.aspx

- Benmoussa, S., El Idrissi, A., Aghoutane, A., & El Hmamouch, F. (2021). Methods of condition monitoring and fault detection for electrical machines. *Energies*, *14*(22), 7459.
- https://doi.org/10.3390/en14227459: https://doi.org/10.3390/en14227459

- Arduino. (n.d.). Get started with machine learning on Arduino. https://docs.arduino.cc/tutorials/nano-33-ble-sense/get-started-with-machine-learning:

- Aslanov, J. (2019). Acoustic event detection [Bachelor's thesis, Czech Technical University in Prague, Faculty of Electrical Engineering, Department of Measurement]. DSpace at Czech Technical University in Prague. https://dspace.cvut.cz/bitstream/handle/10467/86096/F3-BP-2020-Aslanov-Jamal-Acoustic%20Event%20Detection.pdf?sequence=-1&isAllowed=y